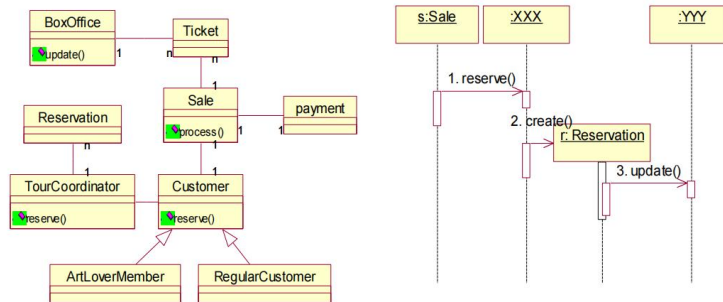


1. 下图所示是某订票系统的类图（部分）和顺序图（部分），其中顺序图中缺少了两个类名，用 XXX 和 YYY 代替，分别可以是（ ）。
- XXX = TourCoordinator, YYY = sale
 - XXX = Customer, YYY = BoxOffice
 - XXX = BoxOffice, YYY = TourCoordinator
 - XXX = TourCoordinator, YYY = BoxOffice



订票系统的类图和顺序图

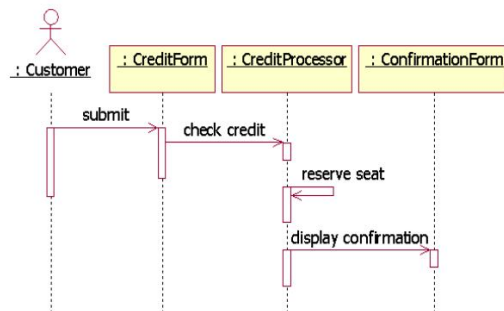
答案：D

分析：

从顺序图中可以看到，`s:Sale` 对象（销售实例）调用了 `XXX` 对象的 `reserve()` 方法。然后，`XXX` 对象创建了 `r:Reservation` 对象（预订实例）。结合类图可以发现，**TourCoordinator**（旅游协调员）有 `reserve()` 方法，且与 **Reservation** 之间存在一条 1 对 1 的关联关系。所以可以推断出 `XXX` 是 **TourCoordinator**。

`r:Reservation` 对象调用了 `YYY` 对象的 `update()` 方法。结合类图发现，只有 **BoxOffice**（售票处）类定义了一个 `update()` 方法。所以 `YYY` 是 **BoxOffice**。

2. 关于下面顺序图正确的说法是（ ）。
- 类 **CreditProcessor** 是控制类，因此它向外发出很多消息。
 - 类 **CreditProcessor** 必须实现 `checkCredit()`，`reserveSeat()` 和 `displayConfirmation()` 方法。
 - CreditForm** 为边界类。
 - ConfirmationForm** 为实体类。



答案：A,C

分析：A. **CreditProcessor** 的确是控制类（Control Class）。且它向外发送多条消息（`reserveSeat`，`displayConfirmation`），所以 A 正确。

B. `displayConfirmation()` 由 **ConfirmationForm** 来实现，而不是 **CreditProcessor**。所以 B 错误。

C. **CreditForm** 是系统与（通常是参与者 Actor）外部交互的接口，属于边界类，C 正确。

D. **ConfirmationForm**（确认表单）和 **CreditForm** 一样，是用来向用户显示信息的界面。因此，它也应该是一个边界类，而不是实体类。所以 D 错误。

1. 根据下面代码，类 HouseKeeper 和类 TimeCard 之间存在（ ）关系。

```
Public class HouseKeeper {  
    public void clockIn( TimeCard timecard){  
    }  
}
```

- A. 包含 (Inclusion)
- B. 依赖 (Dependency)
- C. 关联 (Association)
- D. 泛化 (Generalization)

答案：B

分析：HouseKeeper 类有一个方法 clockIn()，这个方法需要一个 TimeCard 类型的对象作为参数才能执行。这种一个类在某个方法的操作中使用另一个类作为参数、局部变量或返回值，但并不持久地持有它，这就是依赖关系 (Dependency)。

2. 对 Car 和 Engine 两个类关系的实现最可能是（ ）。

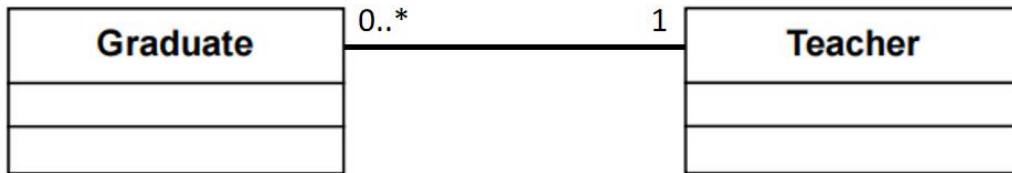
- A. Engine 类中定义一个 Car 类型的属性
- B. 一个 CarEngine 类中定义了一个 Car 类型的属性，一个 Engine 类型的属性
- C. Car 类中定义一个 Engine 类型的属性
- D. Engine 类中定义一个 Car 类型的属性，Car 类中定义一个 Engine 类型的属性

答案：C

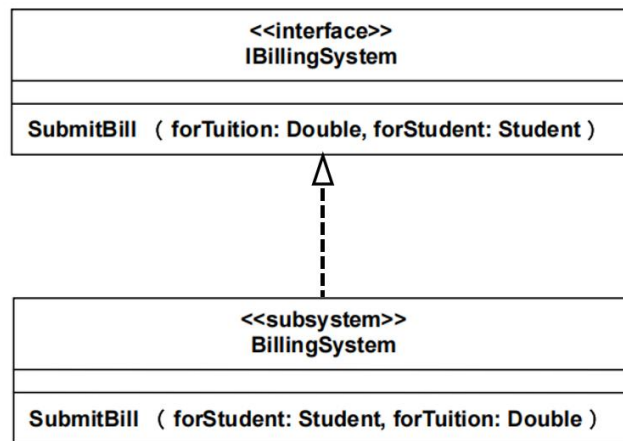
分析：一辆汽车拥有一个发动机 ("has-a"关系)。发动机是汽车的一个组成部分。

1. 请根据下面的描述识别相关元素之间的关系，并在图中使用 UML 符号完整准确地画出这些关系（关联和聚合关系需要标明多重性）。

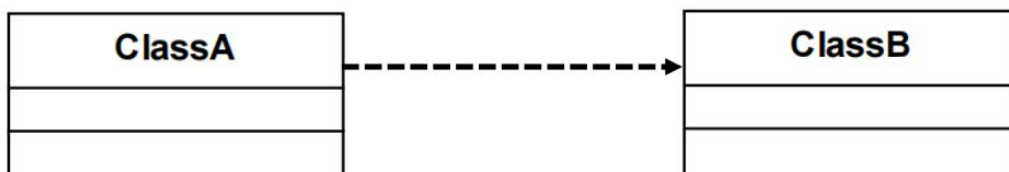
- (1) 在大学中，每一个研究生都有一个指导教师。其中，有些教师指导多个研究生，有些教师不指导任何研究生。



- (2) **BillingSystem** 是学生选课系统的一个子系统，**IBillingSystem** 是该子系统的一个接口。



- (3) 类 **ClassA** 的一个操作调用类 **ClassB** 中的一个操作，除此之外，两个类之间没有任何关系。



2. 请结合下面的描述，建立相关的类图，要求给出相关的类、属性和关系。

After the user has selected items to purchase and then order the items. The user will provide payment and shipping information. The system will respond with confirmation of the order and a tracking number that the user can use to check on order status in the future. The system will also provide the user with an estimated delivery date for the order, which will include all selected items. The user may already have an account with the company with billing and shipping information.

分析： 一共设计七个类，分别是 **User** (用户)、**Account** (账户)、**Order** (订单)、**LineItem** (订单项)、**Item** (商品)、**ShippingInfo** (运输信息) 和 **BillingInfo** (账单信息)。

User 类有 **id** 和 **name** 属性。**Order** 类有 **paymentInfo** (支付信息)、**trackingNumber** (物流追踪号)、**orderStatus** (订单状态) 和 **estimatedDeliveryDate** (预计送达日期) 属性。**LineItem** 类有 **quantity** (数量) 属性。**Item** 类有 **id**、**name** 和 **price** 属性。**ShippingInfo** 类有 **address**、**phoneNumber** 和 **recipient** (收件人) 属性。**BillingInfo** 类有 **id**、**time**、**price** 和 **items** 属性。

User 和 **Account** 之间 (关联)：1 对 0..*

User 和 **Order** 之间 (关联)：1 对 0..*

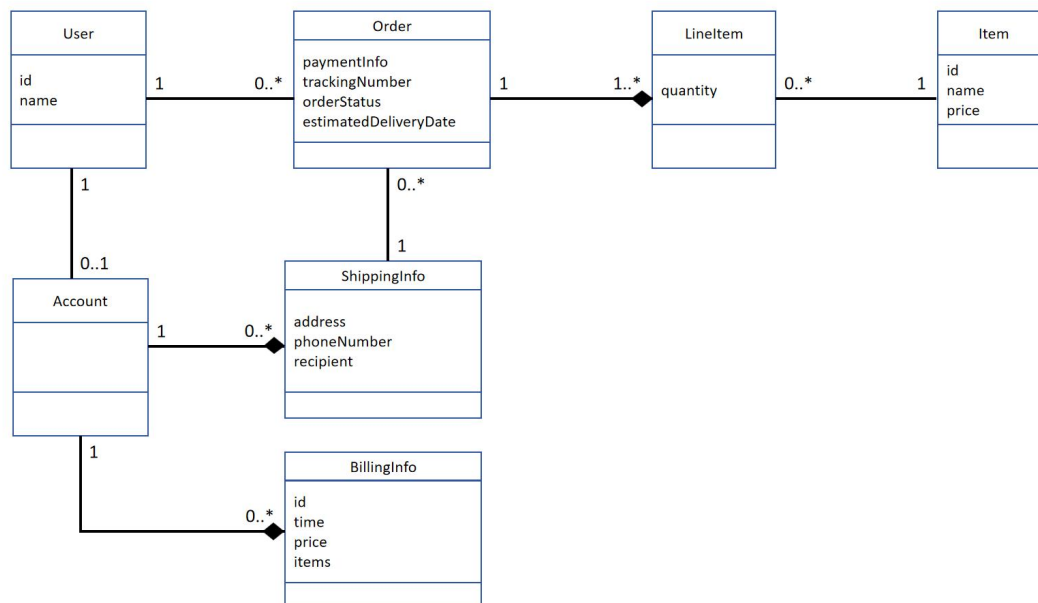
Order 和 **LineItem** 之间 (组合)：1 对 1..*

LineItem 和 **Item** 之间 (关联)：0..* 对 1

Order 和 **ShippingInfo** 之间 (关联)：0..* 对 1

Account 和 **ShippingInfo** 之间 (组合)：1 对 0..*

Account 和 **BillingInfo** 之间 (关联)：1 对 0..*



3. 请结合下面的描述，建立ATM机的状态图。

ATM is initially turned off. After the power is turned on, ATM performs startup action and enters Self Test state. If the test fails, ATM goes into Out of Service state, otherwise there is triggerless transition to the Idle state. In this state ATM waits for customer interaction.

The ATM state changes from Idle to Serving Customer when the customer inserts banking or credit card in the ATM's card reader. On entering the Serving Customer state, the entry action readCard is performed. Note, that transition from Serving Customer state back to the Idle state could be triggered by cancel event as the customer could cancel transaction at any time.

分析: ATM 状态有: Off (关机)、Self Test (自检)、Out of Service (服务终止)、Idle (空闲)、Serving Customer (服务客户)

Off -> Self Test (开机自检)

触发事件 (Event): power on (电源开启)。

转换动作 (Action): startup action (执行启动动作)。

Self Test -> Out of Service (自检失败)

监护条件 (Guard): [test fails] (测试失败)。

Self Test -> Idle (自检成功)

监护条件 (Guard): [test passes] (测试通过)。

Idle (空闲状态)

内部活动 (Do Activity): do: waits for customer interaction (活动: 等待客户交互)。

在 Idle 状态时, ATM 会持续执行“等待交互”这个活动。

Idle -> Serving Customer (客户插卡)

触发事件 (Event): card inserted (卡片插入)。

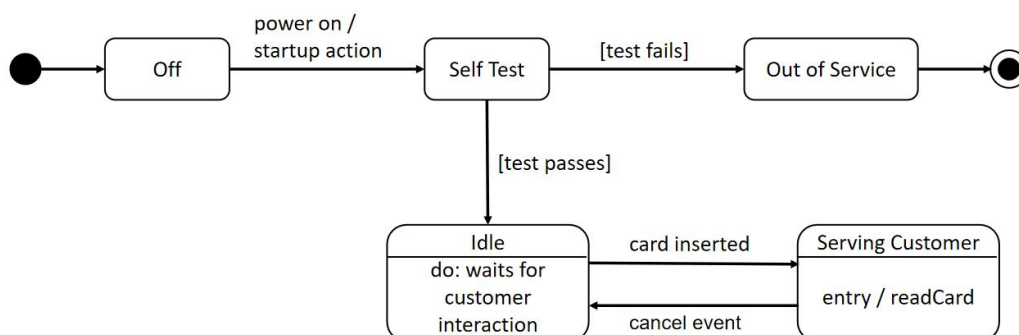
当 ATM 处于 Idle 状态时, 如果客户插入了银行卡, 将触发此事件, 使状态转换为 Serving Customer。

Serving Customer (服务客户状态)

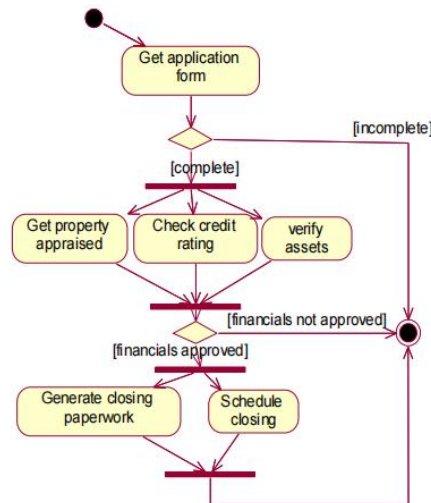
入口动作 (Entry Action): entry / readCard。

Serving Customer -> Idle (客户取消)

触发事件 (Event): cancel event (取消事件)。



1. 根据下面的活动图可推断出，活动（ ）不可并发进行？
- A. Get property appraised、Schedule closing
 - B. Get property appraised、Generate closing paperwork
 - C. Get property appraised、Verify assets、Check credit rating
 - D. Generate closing paperwork、Schedule closing



答案：A,B

分析：Get property appraised、Verify assets、Check credit rating 这三个活动在第三组并发中，Generate closing paperwork、Schedule closing 这两个活动在第二组并发中。第二组并发必须在第一组并发全部完成并且财务状况被批准之后才能开始。所以第一组并发中的活动和第二组的活动不能并发进行，A 和 B 都错误。

2. 下列关于状态图的说法中，正确的是（ ）。
- A. 状态图是 UML 中对系统的静态方面进行建模的五种图之一。
 - B. 状态图是活动图的一个特例，状态图中的多数状态是活动状态。
 - C. 活动图和状态图是对一个对象的生命周期进行建模，描述对象随时间变化的行为。
 - D. 状态图强调对有几个对象参与的活动过程建模，而活动图更强调对单个反应型对象建模。
 - E. 对反应型对象建模一般使用状态图。

答案：E

分析：A 错误。状态图是 UML 中对系统动态行为方面进行建模的图。

B 错误。状态图和活动图是两种不同的图。

C 错误。状态图是对一个对象的生命周期进行建模。而活动图建模的是一个过程或工作流，这个过程可能涉及多个对象。

D 错误。活动图强调对有多个对象参与的活动过程建模；而状态图强调对单个对象建模。

E 正确。反应型对象 (Reactive Object) 指的是那些其行为主要由外部事件驱动的对象（例如：订单对象、UI 按钮、网络连接）。状态图正是用来描述一个对象如何响应各种事件，从而在不同状态之间转换的，因此是为反应型对象建模的理想工具。